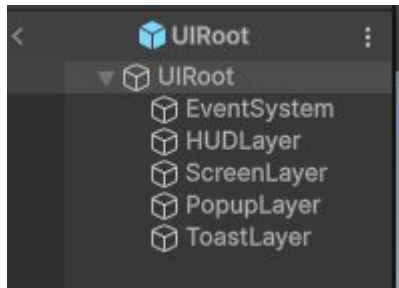


영웅호걸 1차 코드리뷰

- UI Layer 구조 및 자원 관리 -

UI 프레임워크



- UI는 씬에 배치하지 않고 프리팹으로 관리합니다.
- HUD, Screen, Popup, Toast 레이어로 분리하여 렌더링 순서를 관리합니다.
- UIRoot에는 UIManager를 소유하게 하고 하위 layer들을 관리합니다.

UIManager : 싱글톤으로 구현. UI 생성 및 삭제를 한 곳에서 관리하여 디버깅이나 관리를 효율적으로 하기 위한. 최초 생성 이후 DontDestroyOnLoad를 유지. UI의 Open, Close 제어도 Manager가 담당

UIView : UI의 Base 클래스, 어떤 레이어인지를 저장하고 Unity의 Object 생명주기와 별개로 UI의 생명주기를 관리 (UIScreen, UIPopup, UIHud는 UIView를 상속)

UIHud는 체력, 미니맵 등 게임 플레이 중 계속 노출되는 요소를 위한 타입

UIScreen은 전체 화면 성격의 UI로, Title이나 Option, Loading UI 등으로 활용할 예정. 한 번의 하나의 UIScreen만 띄울 수 있다.

UIPopup : 여러 개를 순서대로 중첩할 수 있고 스택을 이용해 가장 최근에 열린 팝업 순서대로 'esc' 명령이 들어오면 닫게 할 것.

UIToast : 는 아직 구현을 못했는데, "네트워크 연결이 끊겼습니다." 같은 잠깐 떴다 사라지는 알림용

Resources 폴더 사용 최소화하기

```
public class UIManager : MonoBehaviour
{
    private const string RootPrefabPath = "UI/UIRoot";
    private const string ViewPathPrefix = "UI/";

    private static UIManager instance;

    public static UIManager Instance
    {
        get
        {
            if (instance != null) return instance;

            UIManager prefab = Resources.Load<UIManager>(RootPrefabPath);
            if (prefab == null)
            {

```

초기 UI Manager 코드의 일부입니다. Resources 폴더를 사용하여 UI 폴더 내부의 프리팹에 접근하고 있습니다!!!

저희 같은 2개월짜리 소규모 프로젝트에서는 상관없지만, 그래도 이왕 하는거 좋은 구조를 쓰고 싶습니다.

Resources 폴더는 빌드 크기를 증가시키고 불필요한 메모리 소모가 생길 수 있습니다. Resources 폴더 내의 모든 리소스가 게임 실행 시 빌드 패키지에 무조건 포함되기 때문이죠! 쓰다가 안쓰게 된 에셋이 있으면 그것도 빌드해버립니다._;

또한 라이브 서비스 중 특정 에셋만 수정하여 빌드할 때에도 Resources에 있는 리소스는 앱 전체를 업데이트해야 합니다. 모바일 게임 중에서도 playstore에서 업데이트 하는거랑 앱 내부에서 서버에 연결하여 간단히 설치만 하는거랑 분리되는 게임들이 있는데 그런 느낌으로 보시면 됩니다.

그래서 대안으로 생각한 것이 Addressables입니다.

- 문자열 대신 Addressable Asset Reference 사용: `AssetReferenceT<T>` 를 스크립터블 오브젝트(SO)나 에셋 설정 파일에 노출하여 에셋을 직접 드래그앤드롭 연결합니다.
- 비동기 로딩 및 리모트 패치 지원: 서버에서 UI 리소스를 다운로드받거나, 앱 업데이트 없이 UI 패치가 가능합니다.

